AIML 231 — Week 5

Dimensionality Reduction

and

Feature Selection

Dr Qi Chen

School of Engineering and Computer Science

Victoria University of Wellington

Qi.Chen@vuw.ac.nz

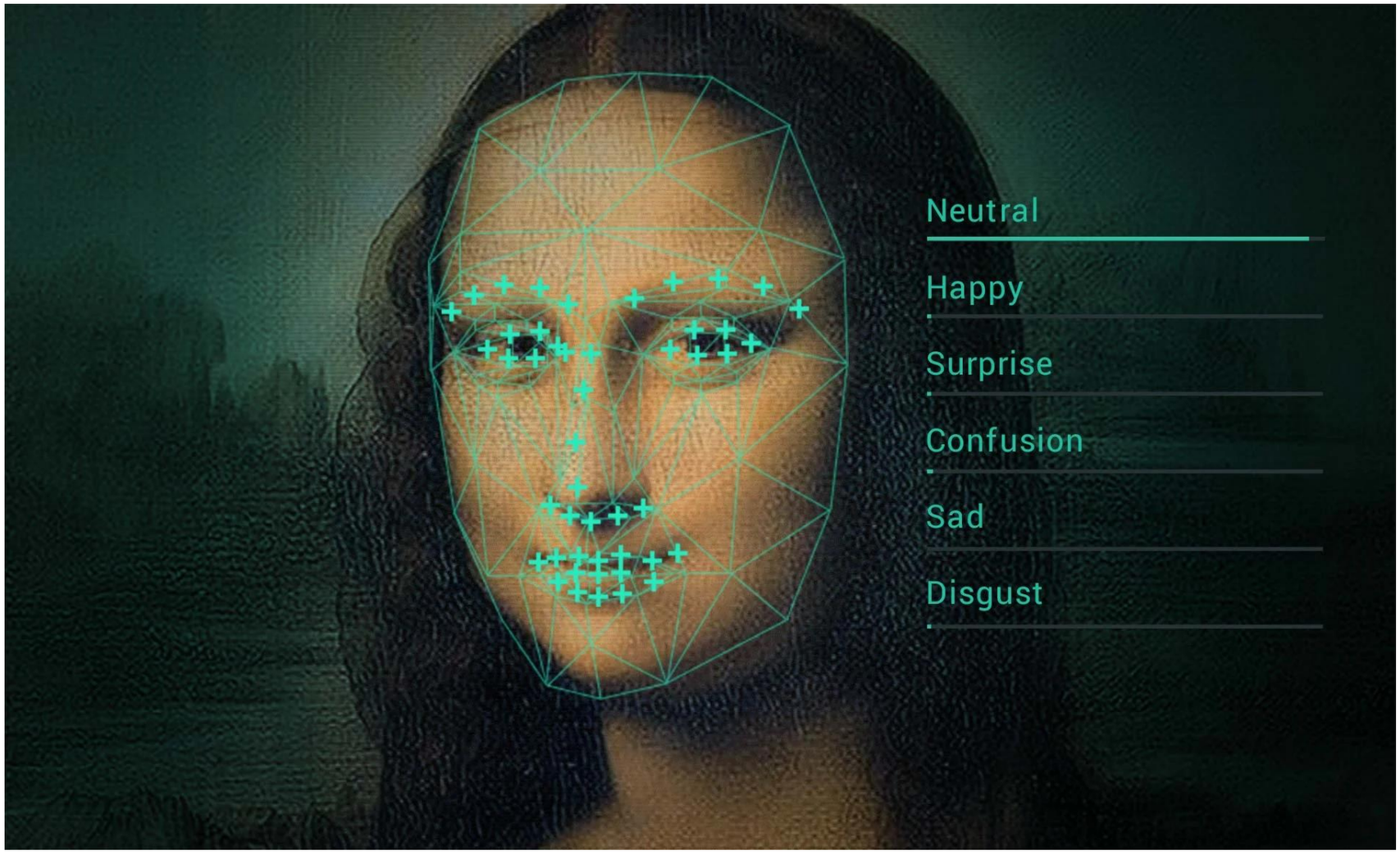
Week Overview

- Introduction of Data Preparation/Preprocessing
  - What data preparation include
  - Why data preparation
  - Types of data preparation techniques

- Data Preparation Techniques
  - Training vs Testing, k-fold cross validation
  - Categorical Data Encoding
  - Normalisation
  - Discretisation

- Dimensionality Reduction
  - Feature Selection
  - Feature Construction

A Motivation



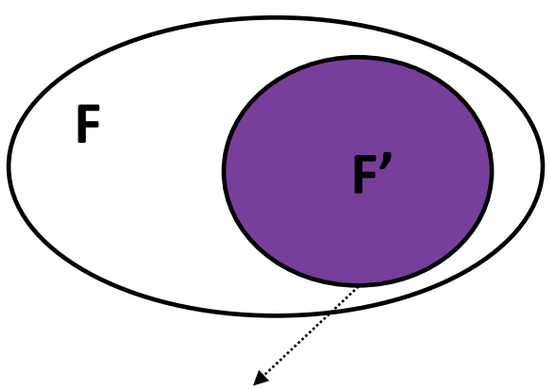
- Ever wonder how face recognition works on your phone?
- Instead of using all pixels, the system identifies and focuses on key features (e.g., eyes, nose, mouth, etc.) that are unique to your face.

Feature Selection and Feature Construction

Feature selection (FS)

- Select a subset of relevant features to achieve similar or better performance than using all features

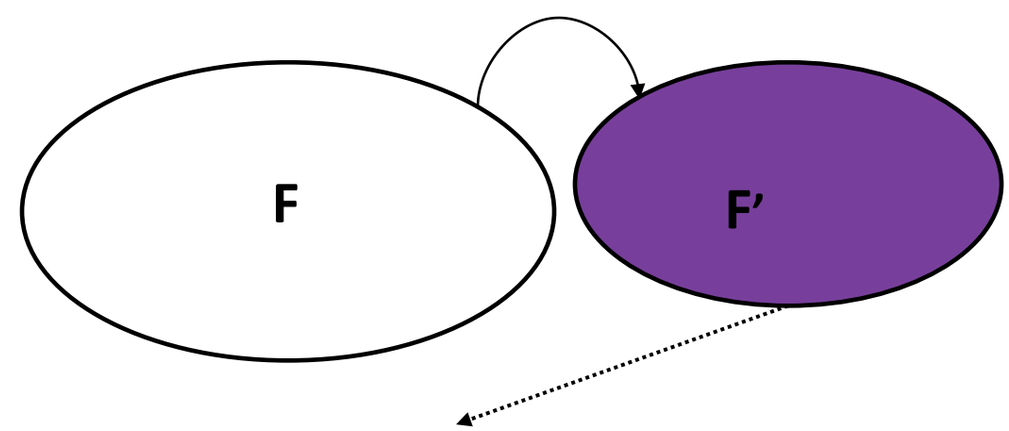
$\mathbf{F} = \{f_1, f_2, \ldots, f_n\}$



$\mathbf{F'} = \{f_{i1}, f_{i2}, \ldots, f_{im}\}$ $(m<n)$

Feature construction (FC)

- Build new high-level features using the original features to improve the classification performance



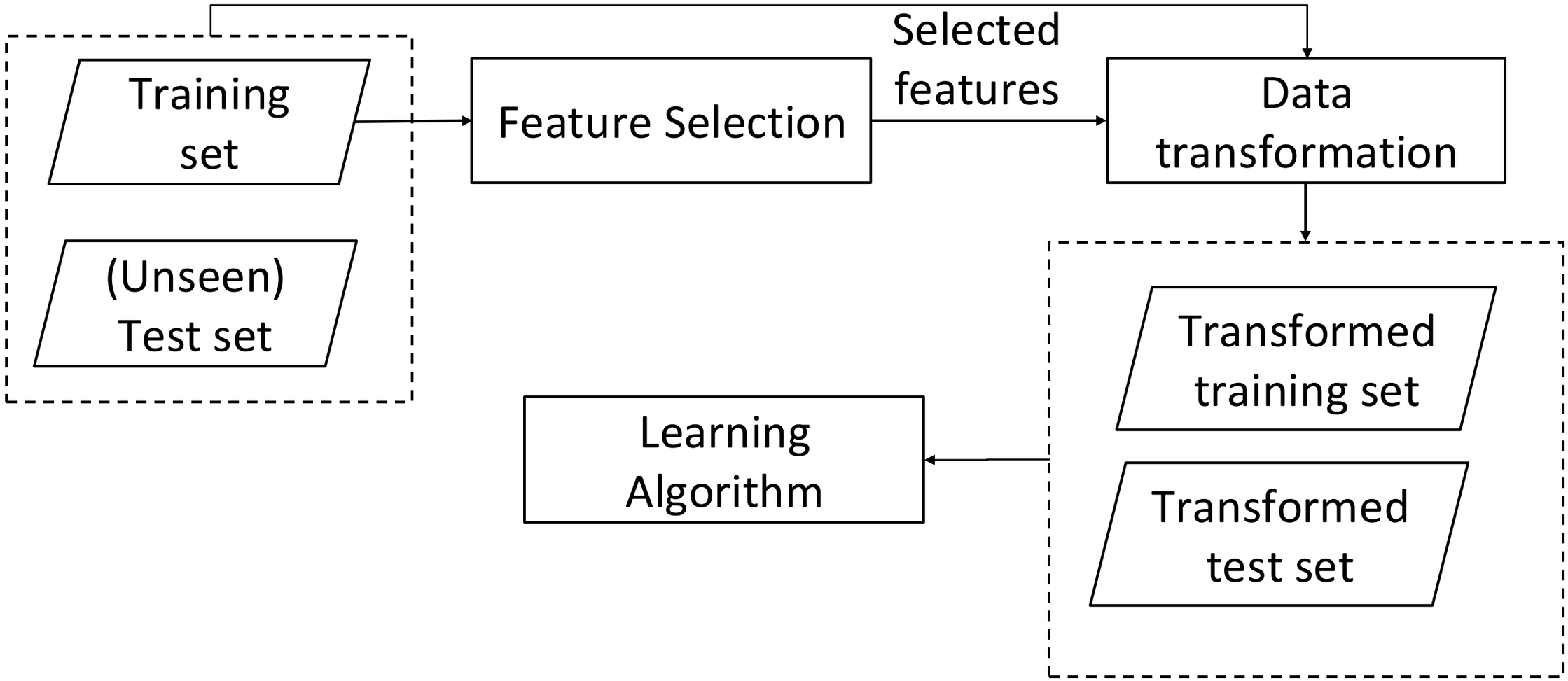
$\mathbf{F'} = \{g_1(f_1,\ldots, f_n), g_2(f_1, \ldots, f_n), \ldots, g_m(f_1, \ldots, f_n)\}$

Feature Selection

- Feature selection is the process of identifying a minimal set of relevant features from the original dataset.
  - Potential trade-off between *number of selected features* and *learning performance*.

- Benefits:
  - Enhances learning performance and interpretability.
  - Reduces the risk of overfitting.
  - Lowers computational and memory costs.

Feature Selection System



- Feature selection is performed only on the training set.
- The transformed training and test sets include only the selected features.
- The testing performance of models trained on the transformed training set and evaluated on the test set reflects the effectiveness of the selected features.

Feature Selection Categories

- Two main approaches:
  - Univariate Feature Selection (Feature Ranking)
    - Evaluates individual features independently.
  - Multivariate Feature Selection (Feature Subset Selection)
    - Considers feature interactions to select the best subset.

Univariate Feature Selection

- Univariate methods run *score functions* to evaluate each feature independently.
  - Relevance: strong relationships with the *target outputs* are good!
  - Redundancy: strong relationships with *other input features* are bad!
- Based on the computed scores, features are ranked, and a predefined number of top-ranked features are selected.
- Key Functions in *sklearn.feature_selection*:
  - *SelectKBest* – Selects the *top K* highest-scoring features.
  - *SelectPercentile* – Selects the *top X%* of features.
  - We'll review some of the score functions you can use in *sklearn.feature_selection*

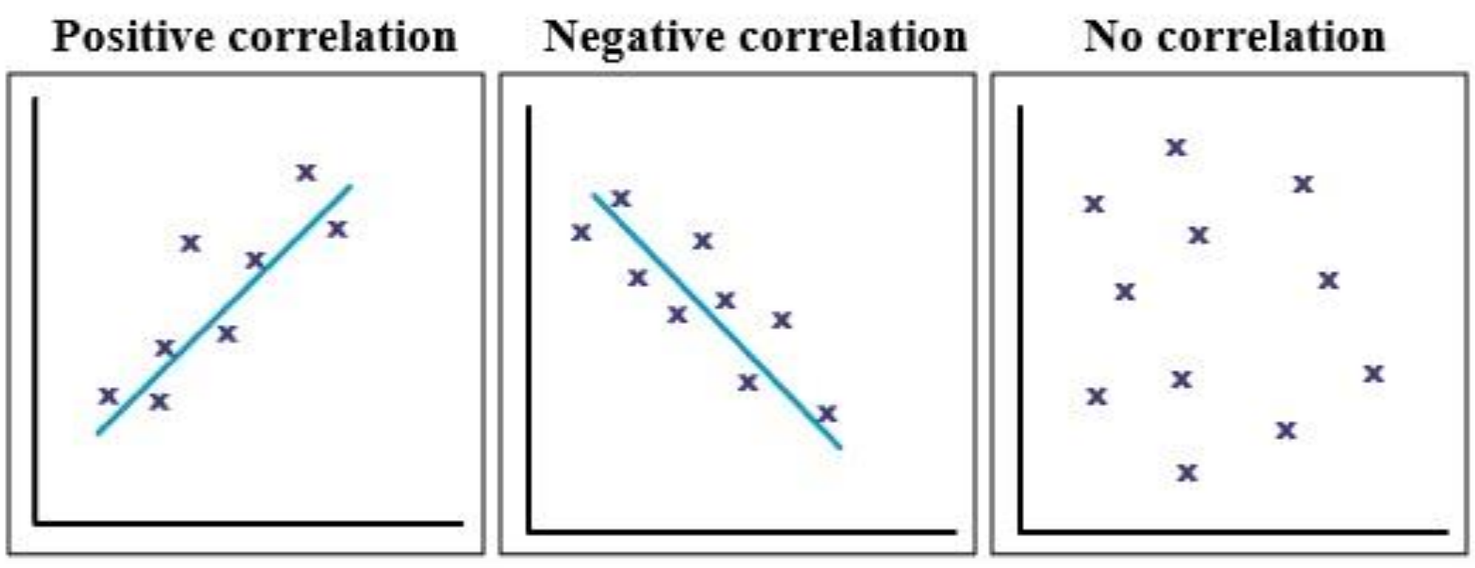
Univariate Feature Selection for Regression (1)

sklearn.feature_selection.r_regression()

- Measures Pearson correlation between each input feature and the target label.
- Captures linear relationships.

$$\text{Correlation} = \frac{\text{Cov}(x, y)}{\sigma_x \sigma_y}$$

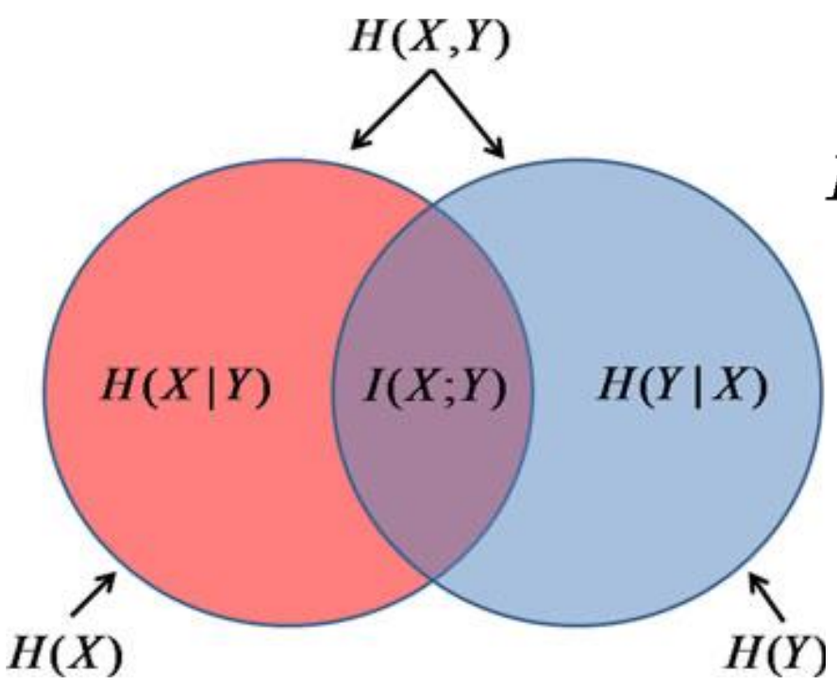
- Produces values in the range [-1, 1], where:
  - +1 indicates a strong positive correlation,
  - -1 indicates a strong negative correlation,
  - 0 means no correlation.
- Features are ranked based on their absolute correlation values (higher values indicate stronger relationships with the target).



Univariate Feature Selection for Regression (2)

sklearn.feature_selection. mutual_info_regression()

- Measures the amount of common information between each input feature and the target variable.
- Captures both linear and non-linear relationships.
- Values are non-negative, with higher values indicating stronger relevance.
- Features are ranked based on their mutual information scores (similar to information gain in decision trees).



$$I(X;Y) = H(X) - H(X|Y) = H(Y) - H(Y|X)$$

Univariate Feature Selection for Classification (1)

sklearn.feature_selection. mutual_info_ classif()

- Similar to *mutual_info_regression(),* but designed for classification tasks with a discrete target lael.
- Measures mutual information between each input feature and the target label.
- Captures both linear and non-linear relationships.
- Values are non-negative, with higher values indicating stronger relevance.
- Features are ranked based on their mutual information scores, just like in *mutual_info_regression().*

Univariate Feature Selection for Classification (2)

sklearn.feature_selection. chi2()

- Called chi-squared.
- Measures the dependence between each categorical features and a discrete target label.
- Features must have non-negative values.
- Returns non-negative values for the Chi-Square statistic, where higher values indicate stronger relevance.

sklearn.feature_selection.r_regression()

- Still can be used directly for binary classification (0/1) class labels.

Univariate Feature Ranking: Code Example

```python
from sklearn.datasets import load_iris
from sklearn.feature_selection import SelectKBest, mutual_info_classif
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load Iris dataset
data = load_iris()
X, y = data.data, data.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Compute mutual information scores
mi_scores = mutual_info_classif(X_train, y_train)

# Select top 2 features based on mutual information
selector = SelectKBest(mutual_info_classif, k=2)
X_train_selected = selector.fit_transform(X_train, y_train)

# Train and test model
clf = RandomForestClassifier(random_state=42)
clf.fit(X_train_selected, y_train)
X_test_selected = selector.transform(X_test)
y_pred = clf.predict(X_test_selected)

# Output accuracy
print(f"Accuracy: {accuracy_score(y_test, y_pred) * 100:.2f}%")
```

Multivariate Feature Selection (1)

- Feature Interaction:
  - Selecting individual features may not be ideal because features can interact with each other.
  - The importance of a feature often depends on its relationship with other features.
  - Therefore, evaluating each feature individually may not provide the full picture.

- Multivariate Feature Selection:
  - Multivariate methods evaluate groups of features together, rather than looking at features individually.
  - This allows for a more comprehensive understanding of how features interact and contribute to model performance.

Multivariate Feature Selection (2)

- Multivariate methods are often implemented as ‘wrappers’ around model training processes.
  - Use a ‘wrapped’ learning models (KNN, DTs, SVMs…).
  - The idea is to experiment/generate different subsets of features.
  - For each feature subset, train the wrapped model, and evaluate learning performance.
  - Can be beyond ‘wrappers’, but that is it for now.

Challenges of Multivariate Feature Selection

- Exponential Search Space:
  - For *n* features, the total number of possible feature combinations (subsets) is 2^n.
  - This means that as the number of features increases, the number of combinations grows exponentially.
- Exhaustive Search is impractical:
  - Exhaustive search, which evaluates every possible subset of features, becomes impossible for large datasets.
- Need for Effective Search:
  - Sequential Forward Selection
  - Sequential Backward Selection
  - *sklearn.feature_selection.SequentialFeatureSelector*

Sequential Forward Selection (1)

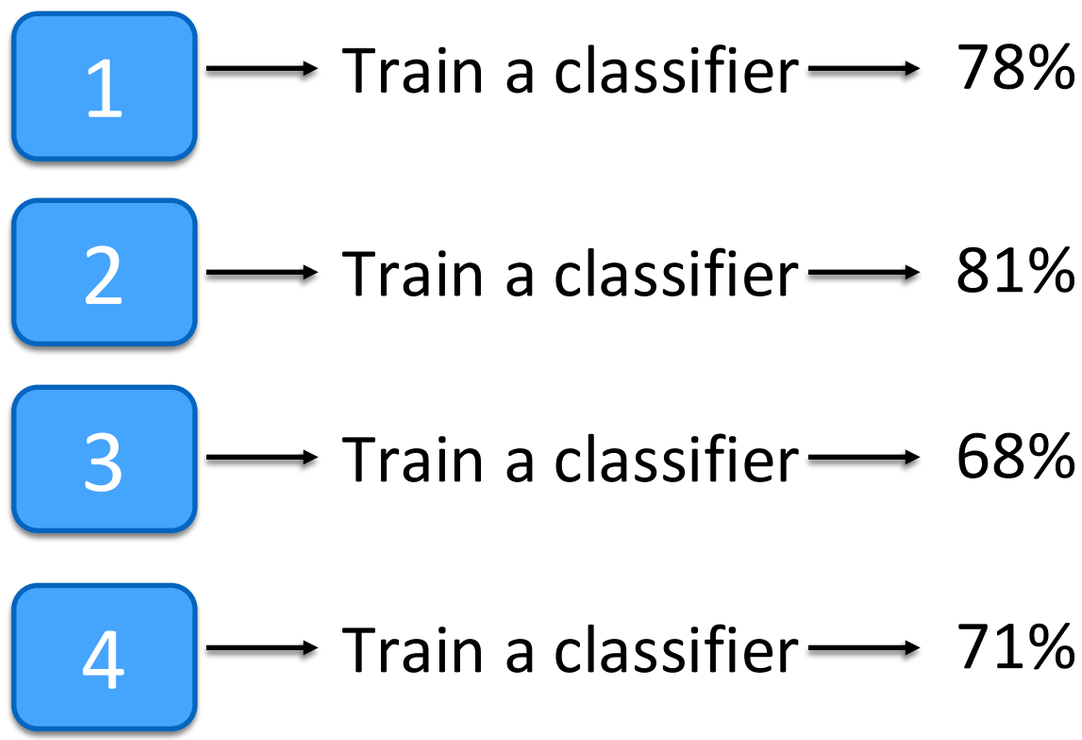
- Starting point:
  - Begin with an empty set of features as the current feature subset.
- Sequential process:
  - Consists of multiple iterations.
  - At each iteration:
    - select the feature *X* that, when added to the current subset, maximises learning performance.
    - Add *X* to the current feature subset.
- Stopping condition:
  - Stop when a pre-defined number of features is selected.

Sequential Forward Selection (2)

Goal: Select 3 features

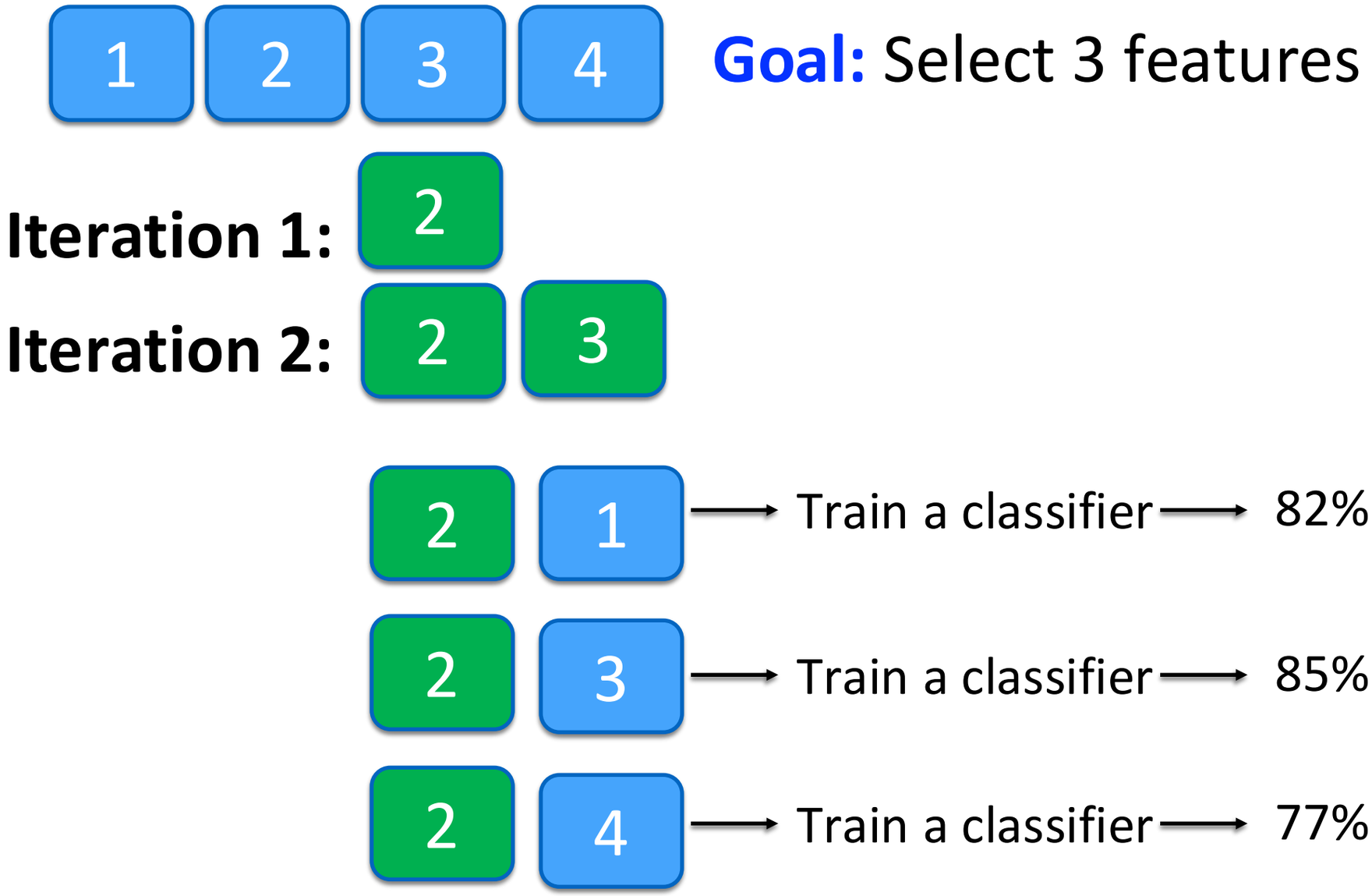
Iteration 0: {}

Iteration 1: 2



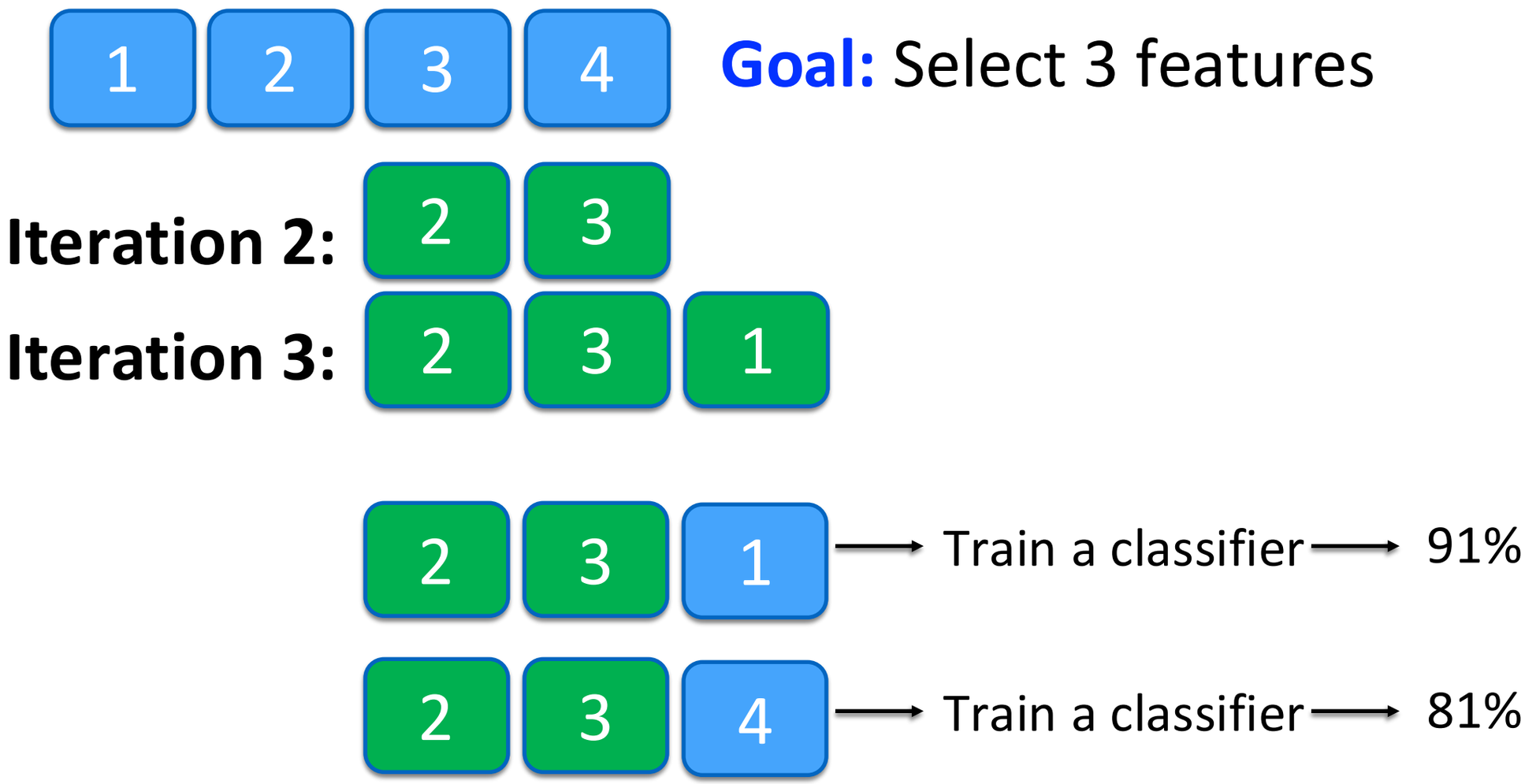
- 4 feature subsets are evaluated.

Sequential Forward Selection (2)



- 3 feature subsets are evaluated.

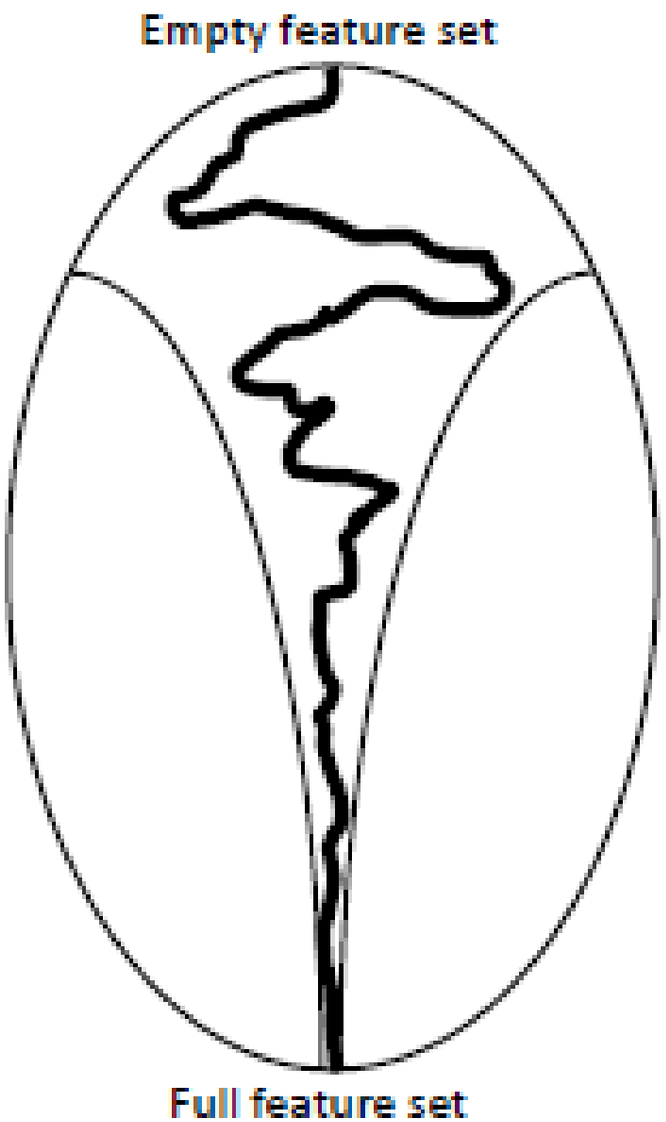
Sequential Forward Selection (2)



- 2 feature subsets are evaluated.

Sequential Forward Selection Discussion

- In our example:
  - Iteration 1: Evaluate 4 feature subsets of size 1.
  - Iteration 2: Evaluate 3 feature subsets of size 2.
  - Iteration 3: Evaluate 2 feature subsets size of 3.
- *Early iterations,* more small feature subsets are explored, as there are many remaining features to consider.
- *Later iterations,* fewer large feature subsets are explored, as there are fewer remaining features to consider.
- Sequential Forward Selection performs best when the optimal feature subset is small.



Sequential Backward Selection (1)

- Starting point:
  - Begin with a full set of features as the current feature subset.
- Sequential process:
  - Consists of multiple iterations.
  - At each iteration:
    - select the feature *X* that, when removed from the current subset, maximises learning performance.
    - remove *X* from the current feature subset.
- Stopping condition:
  - Stop when a pre-defined number of features is selected.

Sequential Backward Selection (2)

1 2 3 4 **Goal:** Select 2 features

Iteration 0: 1 2 3 4

Iteration 1: 2 3 4

1 → Train a classifier on 2 3 4 → 89%

2 → Train a classifier on 1 3 4 → 81%

3 → Train a classifier on 1 2 4 → 71%

4 → Train a classifier on 1 2 3 → 83%

- 4 feature subsets are evaluated.

Sequential Backward Selection (2)

1 2 3 4 **Goal:** Select 2 features

Iteration 1: 2 3 4

Iteration 2: 2 3

2 → Train a classifier on 3 4 → 92%

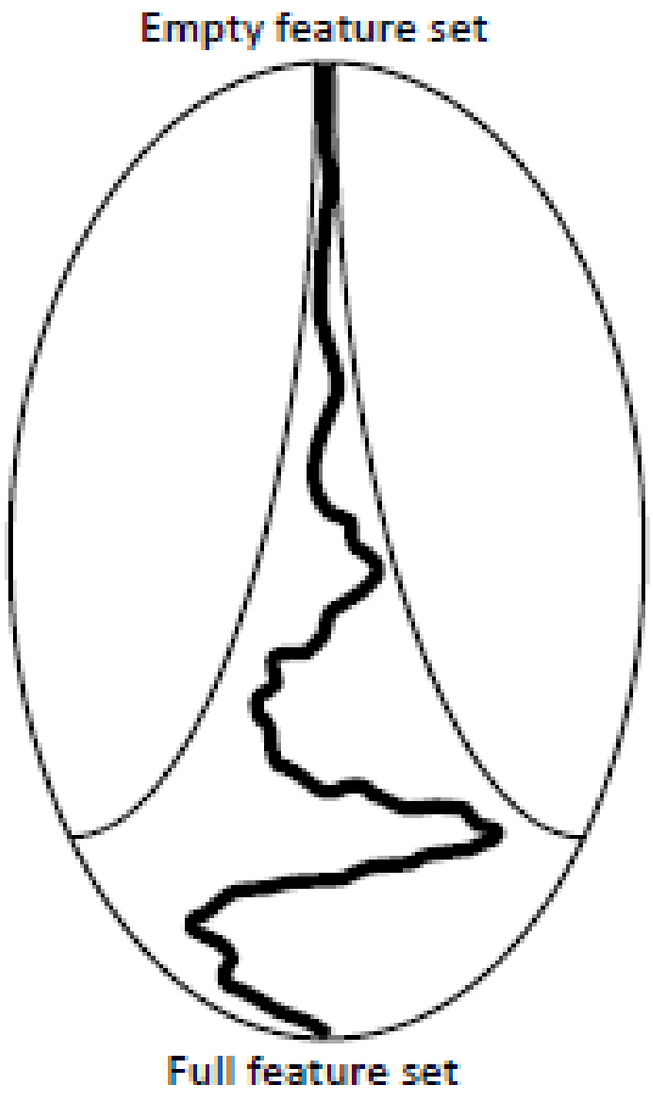
3 → Train a classifier on 2 4 → 89%

4 → Train a classifier on 2 3 → 95%

- 3 feature subsets are evaluated.

Sequential Backward Selection Discussion

- In our example:
  - Iteration 1: Evaluate 4 feature subsets of size 3.
  - Iteration 2: Evaluate 3 feature subsets of size 2.
- *Early iterations,* more large feature subsets are explored, as there are many remaining features to remove.
- *Later iterations,* fewer small feature subsets are explored, as there are fewer remaining features to remove.
- Sequential Backward Selection performs best when the optimal feature subset is large.



Final Remarks

- Univariate Feature Selection/Feature Ranking
  - Simple, efficient.
  - Ignore feature interactions.
- Multivariate Feature Selection/Feature Subset Selection
  - Consider feature interactions, usually better performance.
  - More complicated search, usually more expensive than ranking.

- Is Sequential Search perfect: Not really…
  - Nesting effect:
    - SFS cannot remove features added in previous steps.
    - SBS cannot add features removed in previous steps.
  - Limited searchability.
  - More in higher-level courses, take **AIML427 – Big Data**